

System Identification Project

Part Two: Nonlinear ARX Identification

Professor:

Bușoniu Ioan Lucian

Supervisor:

Sântejudean Tudor Voicu

Students:

Coș Tudor-Adrian

Mastan Andrei-Mircea

Moga Anton-Ioan

Group: 30332

Group index: 2/9

Table of contents

1. Problem statement
2. Methodology
3. Key features
4. Representative plots
5. MSE Representation
6. Conclusion
7. Appendix

Problem statement

- ❖ We study the output y of an unknown dynamic system with a single input and a single output. Two datasets are provided: an identification set and a separate validation set, both measured on the same system.
- ❖ The objective of this project is to build a black-box polynomial nonlinear ARX model that approximates the system's input-output behavior and to use this model to predict and simulate the output for previously unseen validation data.

Methodology

- ❖ Autoregressive with exogenous input (ARX) model is a linear system identification model that expresses the current output using past outputs and external inputs
- ❖ Nonlinear ARX (NARX) model extends ARX by generalizing it to any nonlinear dependence
- ❖ The general mathematical formula of a NARX structure:

$$\hat{y}(k) = [y(k-1), \dots, y(k-na), u(k-1), \dots, u(k-nb)] \cdot \Theta + e(k)$$

- ❖ Examples for different values of na, nb and m:
 - **na=nb=1, m=1** -> $\hat{y}(k) = [1, y(k-1), u(k-1)] \cdot \Theta$
 - **na=nb=1, m=2** -> $\hat{y}(k) = [1, y(k-1), u(k-1), y^2(k-1), u^2(k-1), y(k-1)u(k-1)] \cdot \Theta$
 - **na=nb=1, m=3** -> $\hat{y}(k) = [1, y(k-1), u(k-1), y^2(k-1), u^2(k-1), y(k-1)u(k-1), y^3(k-1), u^3(k-1), y^2(k-1)u(k-1), y(k-1)u^2(k-1)] \cdot \Theta$
 - **na=nb=2, m=2** -> $\hat{y}(k) = [1, y(k-1), y(k-2), u(k-1), u(k-2), y^2(k-1), y^2(k-2), u^2(k-1), u^2(k-2), y(k-1)y(k-2), u(k-1)u(k-2), y(k-1)u(k-1), y(k-1)u(k-2), y(k-2)u(k-1), y(k-2)u(k-2)] \cdot \Theta$

Methodology

- ❖ The regression vector $\Phi(k)$ is built from past outputs and inputs:

$$d(k) = [y(k-1), \dots, y(k-n_a), u(k-1), \dots, u(k-n_b)]$$

- ❖ Phi is generated by expanding $d(k)$ into polynomial terms up to degree m
- ❖ Each row of Phi contains all monomials of $d(k)$ computed earlier
- ❖ Model complexity and accuracy depend on n_a , n_b and the polynomial degree m

Methodology

- ❖ In order to find theta, we use the same method we used in Part One of this project:

$$\hat{\Theta} = (\Phi^T \Phi)^{-1} \Phi^T Y$$

- ❖ We now can calculate the approximated output on the validation set, using theta:

$$\hat{Y} = \Phi \hat{\Theta}$$

- ❖ The approximated output $\hat{Y}$ is evaluated in two ways:
 - Prediction (one-step-ahead): real past outputs are used
 - Simulation: previously simulated outputs are fed back into the model

Key features

❖ We found 2 methods of generating Phi

1. Exponent-table (power-vector) Polynomial Expansion

- Uses a precomputed table of exponents to generate all polynomial terms up to degree m
- Each row of the exponent table defines one monomial
- Phi is built as: $\Phi_k(r) = \prod_i d_i^{\text{pow}(r,i)}$
- Best for higher degrees ($m > 3$), flexibility and performance (avoids loops)

2. Direct Monomial Expansion

- Explicitly constructs monomials using nested loops
- Polynomial terms are generated manually
- Phi grows step by step with degree m
- Best for low-degree models ($m \leq 3$) and clarity

Key features

1. Exponent-table (power-vector) Polynomial Expansion

```
299 % generarea unui tabel de exponenți
300 function pow=powers_generator(p,m)
301     pow=zeros(1,p);
302     for i=1:p
303         e=zeros(1,p);
304         e(i)=1;
305         pow=[pow;e];
306     end
307     if m>=2
308         for i=1:p
309             for j=i:p
310                 e=zeros(1,p); e(i)=e(i)+1; e(j)=e(j)+1;
311                 pow=[pow;e];
312             end
313         end
314     end
315     if m>=3
316         for i=1:p
317             for j=i:p
318                 for k=j:p
319                     e=zeros(1,p); e(i)=e(i)+1; e(j)=e(j)+1; e(k)=e(k)+1;
320                     pow=[pow;e];
321                 end
322             end
323         end
324     end
325 end
```

```
336 % generarea phi cu ajutorul tabelului de exponenți
337 % mai bun pentru alte valori ale lui m
338 function phi_k=powers_phi_arx_generator(y,u,k,na,nb,pow)
339 |
340     d=zeros(1,na+nb);
341     for i=1:na
342         if (k-i)>0
343             d(i)=y(k-i);
344         end
345     end
346
347     for j=1:nb
348         if (k-j)>0
349             d(na+j)=u(k-j);
350         end
351     end
352
353     phi_k=zeros(1,size(pow,1));
354     for r=1:size(pow,1)
355         phi_k(r)=prod(d.^pow(r,:));
356     end
357
358 end
```

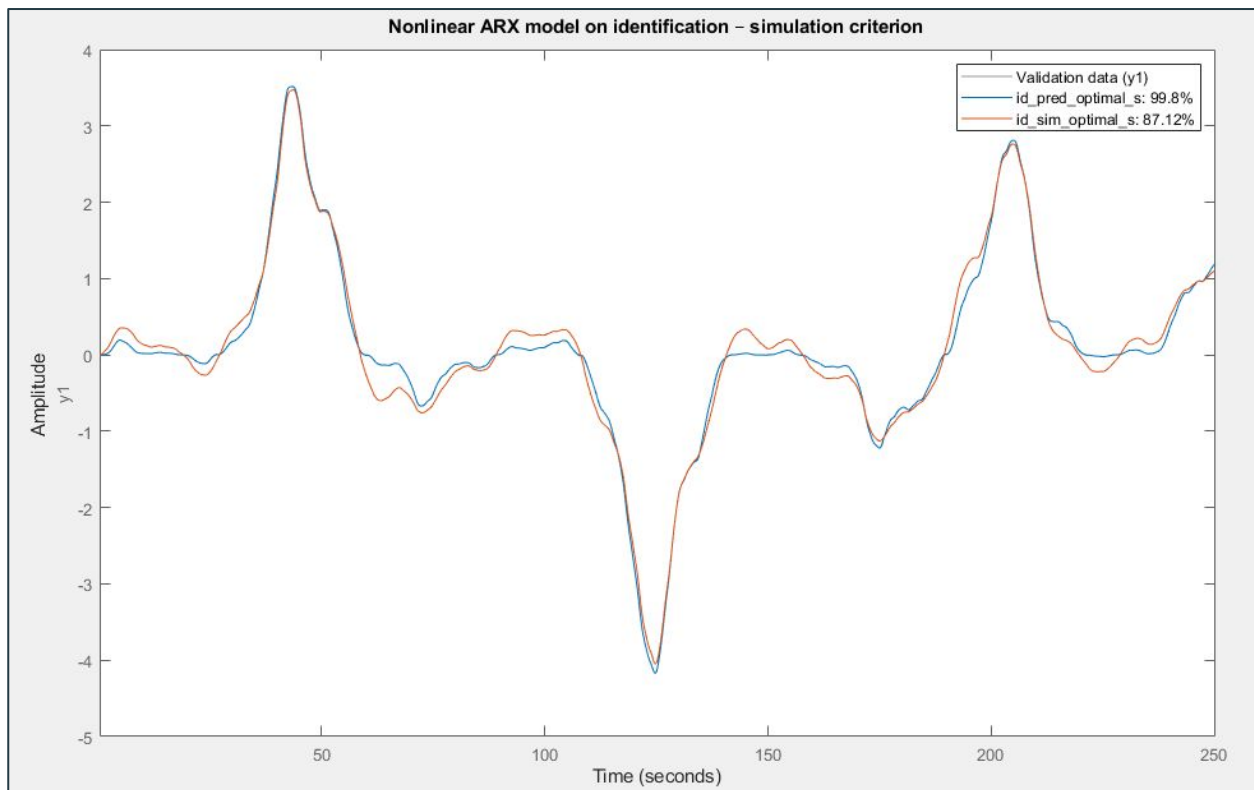
Key features

2. Direct Monomial Expansion

```
263 function phi_k=phi_arx_generator(y,u,k,na,nb,m)
264
265     d=zeros(1,na+nb);
266     for i=1:na
267         if k-i>0
268             d(i)=y(k-i);
269         end
270     end
271
272     for j=1:nb
273         if k-j>0
274             d(na+j)=u(k-j);
275         end
276     end
277     phi_k=[1 d];
278
279     if m>=2
280         for i=1:length(d)
281             for j=i:length(d)
282                 phi_k=[phi_k d(i)*d(j)];
283             end
284         end
285     end
```

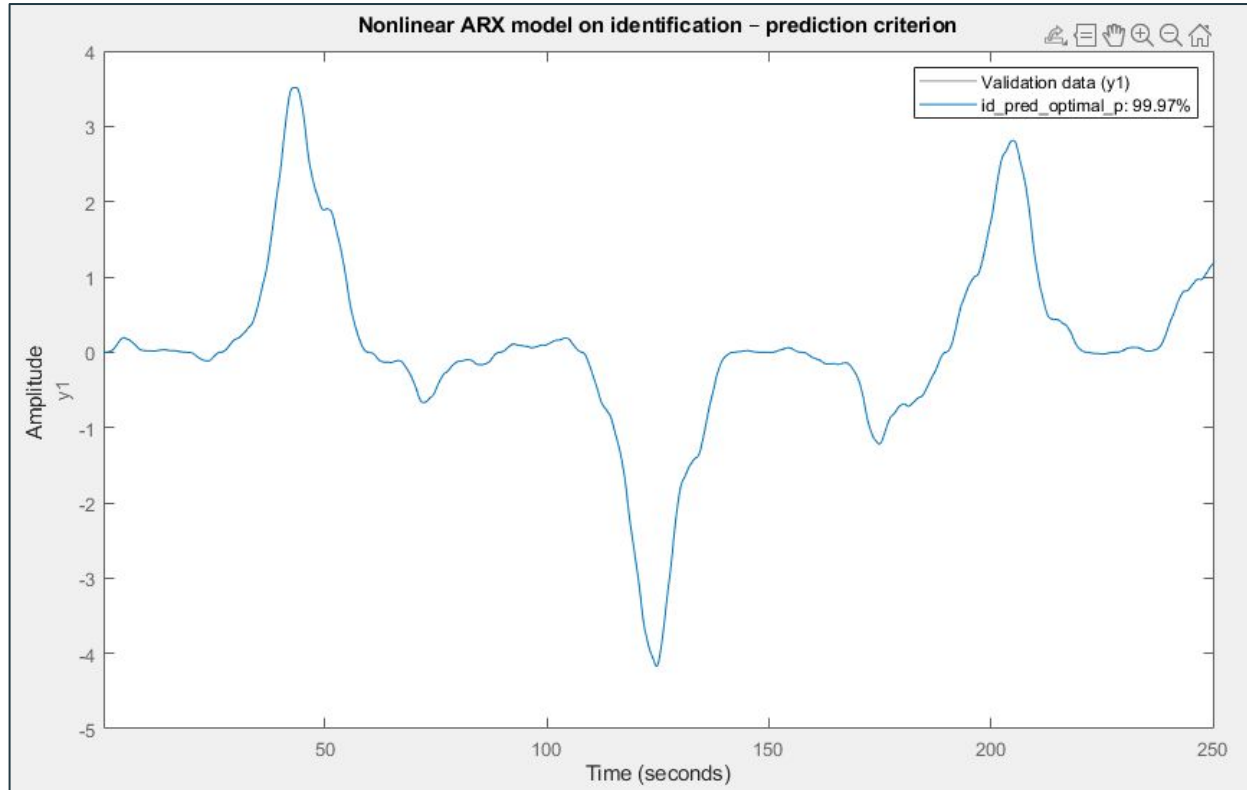
```
287         if m>=3
288             for i=1:length(d)
289                 for j=i:length(d)
290                     for l=j:length(d)
291                         phi_k=[phi_k d(i)*d(j)*d(l)];
292                     end
293                 end
294             end
295         end
296     end
297 end
```

Representative plots



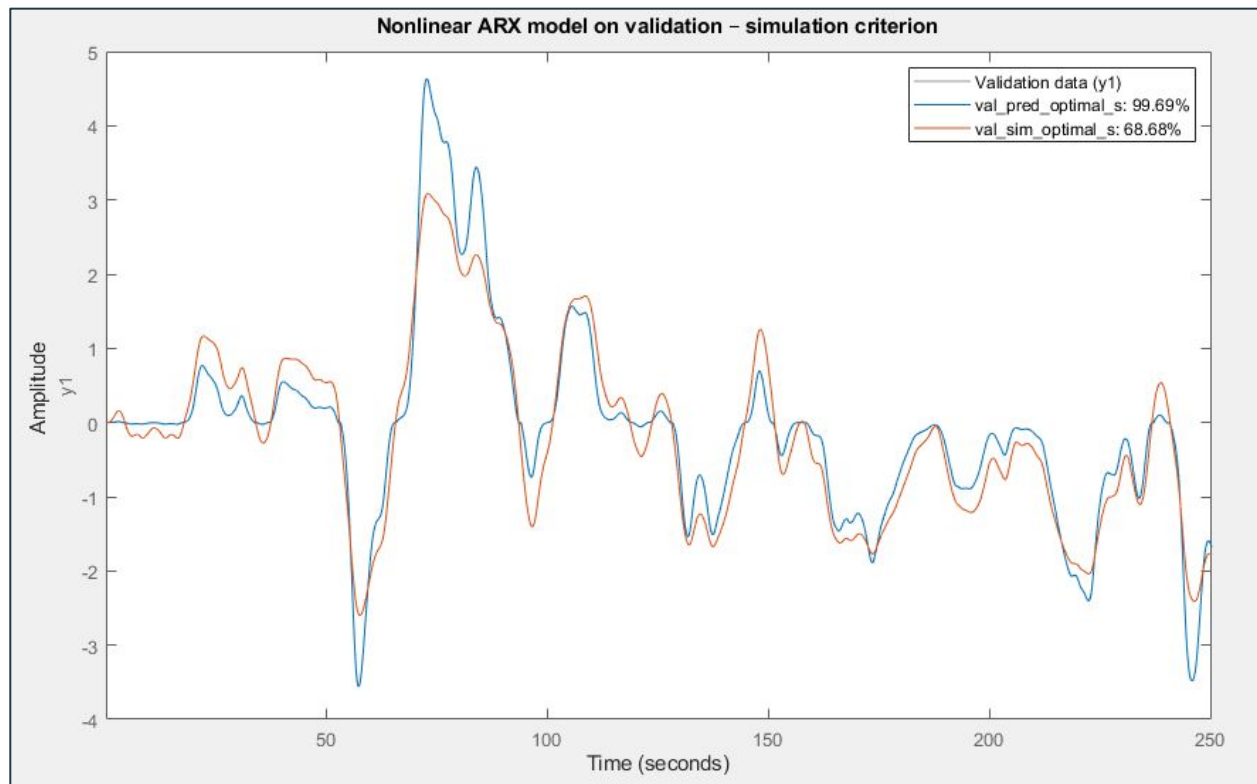
Best simulation model on id:
na = nb = 2
m = 3
MSE = 0.022938

Representative plots



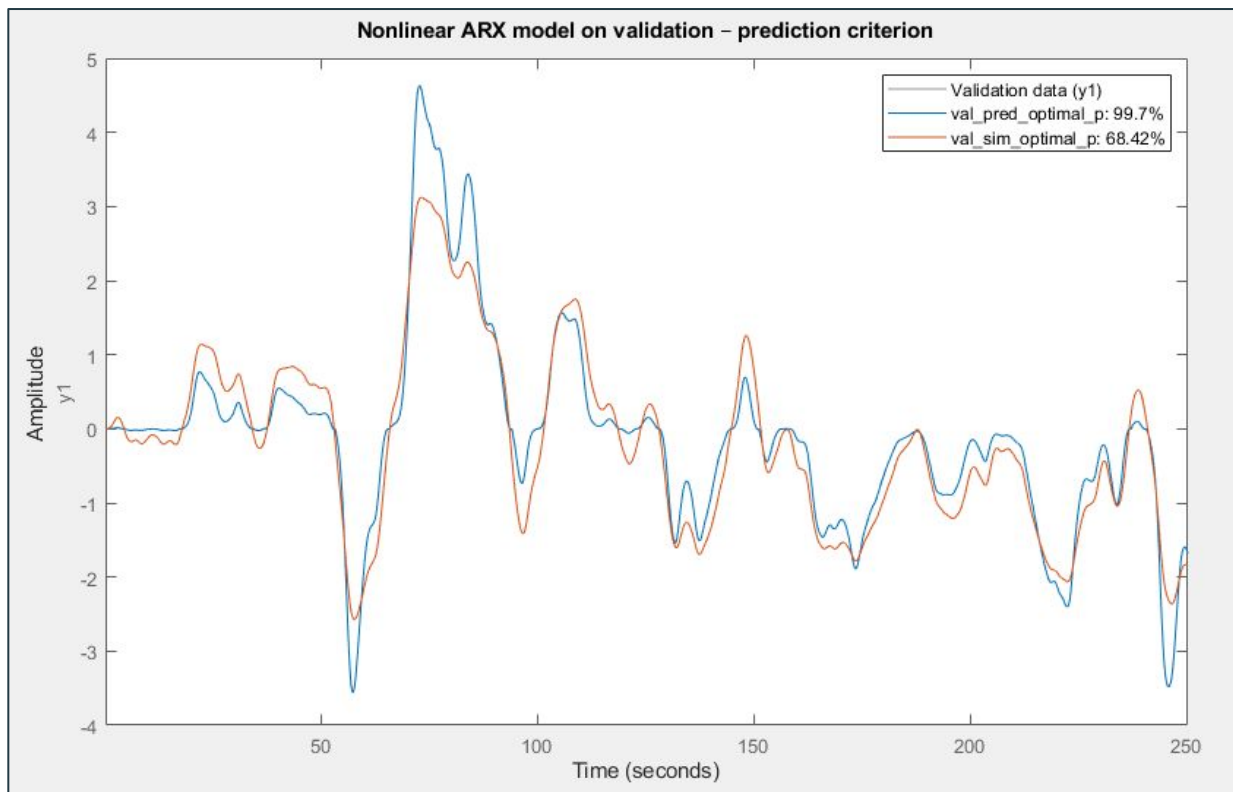
Best prediction model on id:
na = nb = 6
m = 3
MSE = 0.000000

Representative plots



```
Best simulation model on val:  
na = nb = 6  
m = 1  
MSE = 0.161346
```

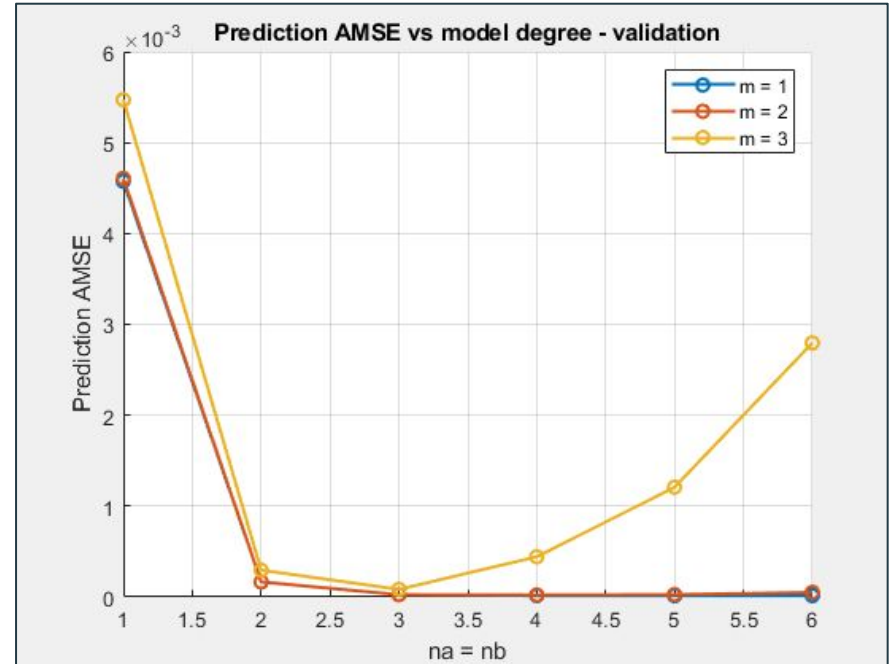
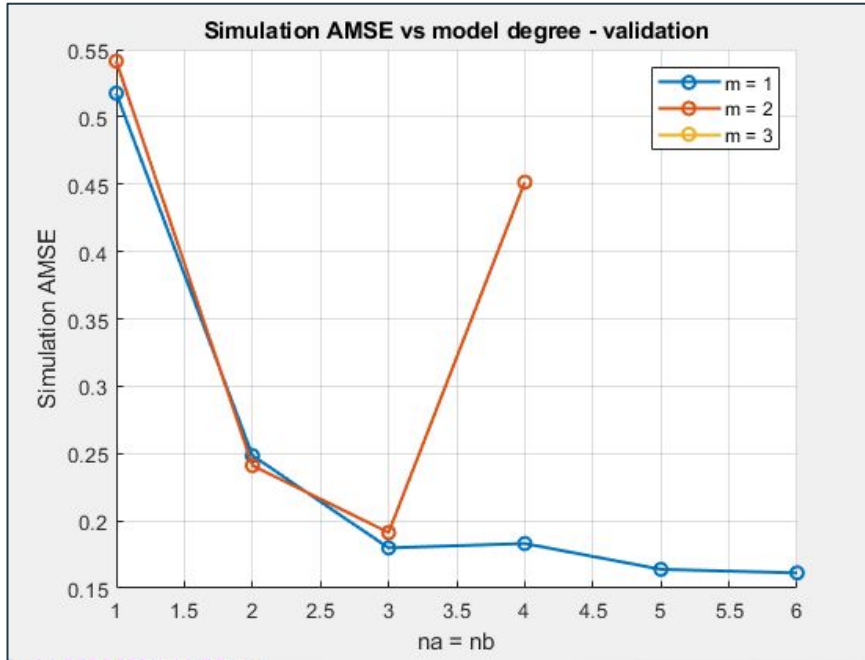
Representative plots



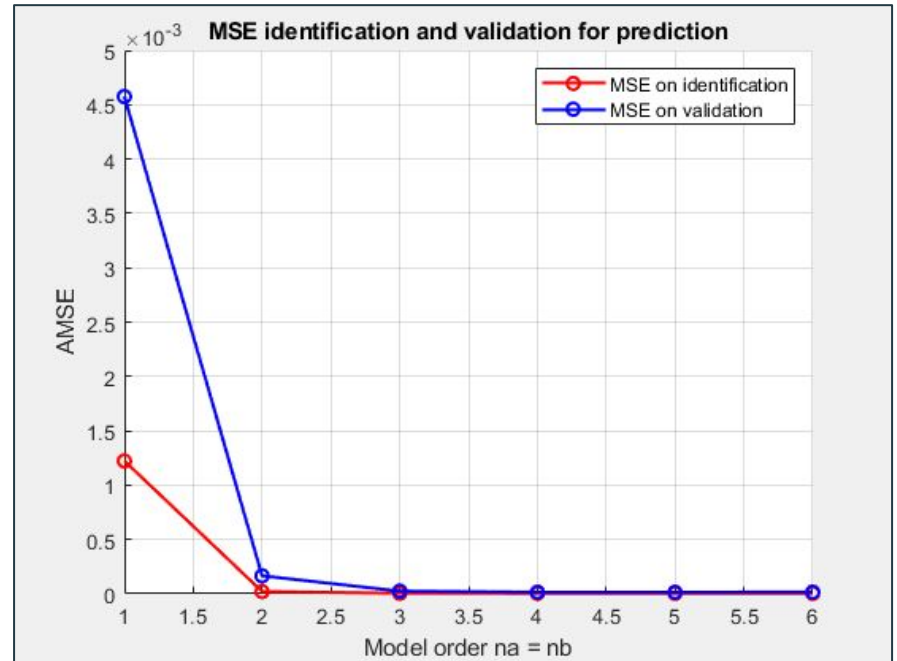
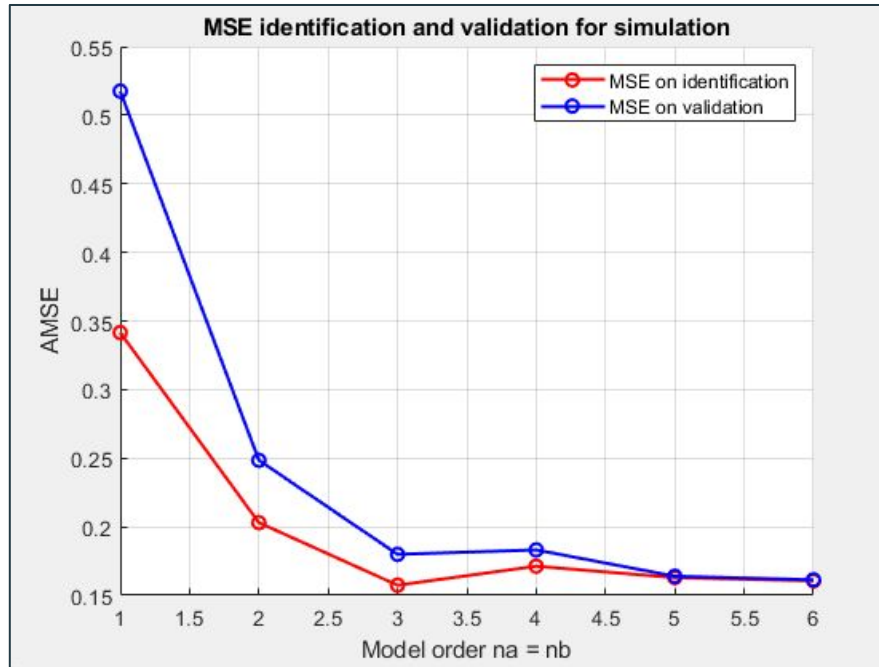
```
Best prediction model on val:  
na = nb = 5  
m = 1  
MSE = 0.000015
```

MSE Representation

- ❖ The formula for computing MSE is:
$$\text{MSE} = (1/N) * \sum_{k=1}^N (y(k) - \hat{y}(k))^2$$



MSE Representation (for optimal case)



Conclusions

- ❖ A polynomial nonlinear ARX black-box model was successfully identified from data
- ❖ Model order selection shows clear underfitting for small orders and overfitting for large ones
- ❖ One-step-ahead prediction performs better than simulation, as expected for nonlinear ARX structures
- ❖ Validation results confirm good generalization for the optimal (n_a, m) combination
- ❖ Comparing identification and validation MSE curves proved essential for selecting a model that generalizes well

Appendix

1	<code>%% project part 2</code>	
2	<code>clc; clear;</code>	
3	<code>load("iddata-09.mat");</code>	
4		
5	<code>%% initializare date si plot</code>	
6		
7	<code>t_sampling=id.Ts;</code>	
8	<code>u_id=id.U;</code>	
9	<code>y_id=id.Y;</code>	
10	<code>u_val=val.U;</code>	
11	<code>y_val=val.Y;</code>	
12		
13	<code>t_id=0:t_sampling:(length(u_id)-1)*t_sampling;</code>	
14	<code>t_val=0:t_sampling:(length(u_val)-1)*t_sampling;</code>	
15		
16	<code>N_id=length(y_id);</code>	
17	<code>N_val=length(y_val);</code>	
18		
19	<code>figure(1);</code>	
20	<code>plot(t_id,u_id);</code>	

21	hold on;	
22	plot(t_id,y_id);	
23	title("Identification data");	
24	legend("input","output");	
25	figure(2);	
26	plot(t_val,u_val);	
27	hold on;	
28	plot(t_val,y_val);	
29	title("Validation data");	
30	legend("input","output");	
31		
32	%%	
33	max_na=6;	
34	max_m=3;	
35		
36	AMSE_id_pred=zeros(max_na,max_m);	
37	AMSE_id_sim=zeros(max_na,max_m);	
38	AMSE_val_sim=zeros(max_na,max_m);	
39	AMSE_val_pred=zeros(max_na,max_m);	
40	% provizorii pana cand na si nb sunt diferite	

```
41
42 for m=1:max_m
43     for na=1:max_na
44         nb=na;
45         pow=powers_generator(na+nb,m);
46
47         % IDENTIFICATION
48         % pt fiecare m avem cate 6 na=nb si facem cate un theta pt fiecare
49         % in total 6*6=36 thetauri
50         Phi_id=[];
51         Y_id=y_id(:);
52         for k=1:N_id
53             phi_k=phi_arx_generator(y_id,u_id,k,na,nb,m);
54             Phi_id=[Phi_id; phi_k];
55         end
56
57         theta=Phi_id\Y_id;
58         y_id_pred=Phi_id*theta;
59
```

```
60 % simulation on id
61 y_id_sim=zeros(N_id,1);
62 for k=1:N_id
63     phi_k=phi_arx_generator(y_id_sim,u_id,k,na,nb,m);
64     y_id_sim(k)=phi_k*theta;
65 end
66
67 % PREDICTION on val
68 %la fel pt fiecare m si cate 5 na=nb avem cate o predictie
69 y_val_pred=zeros(N_val,1);
70 for k=1:N_val
71     phi_k=phi_arx_generator(y_val,u_val,k,na,nb,m);
72     y_val_pred(k)=phi_k*theta;
73 end
74
75 % SIMULATION on val
76 y_val_sim=zeros(N_val,1);
77 for k=1:N_val
78     phi_k=phi_arx_generator(y_val_sim,u_val,k,na,nb,m);
79     y_val_sim(k)=phi_k*theta;
```

```

80         end
81
82         % ERRORS
83         AMSE_id_pred(na,m)=mean((y_id-y_id_pred).^2);
84         AMSE_id_sim(na,m)=mean((y_id-y_id_sim).^2);
85         AMSE_val_pred(na,m)=mean((y_val-y_val_pred).^2);
86         AMSE_val_sim(na,m)=mean((y_val-y_val_sim).^2);
87         % formule alternative pt calcul MSE
88         % AMSE_id_pred(na,m)=(1/length(y_id))*sum((y_id_pred-y_id).^2);
89         % AMSE_id_sim(na,m)=(1/length(y_id))*sum((y_id_sim-y_id).^2);
90         %AMSE_val_pred(na,m)=(1/length(y_val))*sum((y_val_pred(1:length(y_val)))-y_val).^2);
91         %AMSE_val_sim(na,m)=(1/length(y_val))*sum((y_val_sim(1:length(y_val)))-y_val).^2);
92
93         if (m==1&&na==1)
94             min_MSE_val_sim=AMSE_val_sim(na,m);
95             min_MSE_val_pred=AMSE_val_pred(na,m);
96             min_MSE_id_pred=AMSE_id_pred(na,m);
97             min_MSE_id_sim=AMSE_id_sim(na,m);
98         end

```

```
93     if (m==1&&na==1)
94         min_MSE_val_sim=AMSE_val_sim(na,m);
95         min_MSE_val_pred=AMSE_val_pred(na,m);
96         min_MSE_id_pred=AMSE_id_pred(na,m);
97         min_MSE_id_sim=AMSE_id_sim(na,m);
98     end
99
100     if AMSE_id_pred(na,m)<=min_MSE_id_pred
101         min_MSE_id_pred=AMSE_id_pred(na,m);
102         id_pred_optimal_p=iddata(y_id_pred,u_id,t_sampling);
103         id_sim_optimal_p=iddata(y_id_sim,u_id,t_sampling);
104     end
105
106     if AMSE_id_sim(na,m)<=min_MSE_id_sim
107         min_MSE_id_sim=AMSE_id_sim(na,m);
108         id_sim_optimal_s=iddata(y_id_sim,u_id,t_sampling);
109         id_pred_optimal_s=iddata(y_id_pred,u_id,t_sampling);
110     end
111
```

```
112     if AMSE_val_sim(na,m)<=min_MSE_val_sim
113         min_MSE_val_sim=AMSE_val_sim(na,m);
114         val_pred_optimal_s=iddata(y_val_pred,u_val,t_sampling);
115         val_sim_optimal_s=iddata(y_val_sim,u_val,t_sampling);
116     end
117
118     if AMSE_val_pred(na,m)<=min_MSE_val_pred
119         min_MSE_val_pred=AMSE_val_pred(na,m);
120         val_pred_optimal_p=iddata(y_val_pred,u_val,t_sampling);
121         val_sim_optimal_p=iddata(y_val_sim,u_val,t_sampling);
122     end
123
124     end
125 end
```

128	%% IDENTIFICATION	
129		
130	%% Best model on identification (SIMULATION CRITERION)	
131	% (cele mai bune combinatii de na,nb,m dupa MSE_sim)	
132		
133	[minAMSE_id_sim,idx]=min(AMSE_id_sim(:));	
134	[id_best_na_sim,id_best_m_sim]=ind2sub(size(AMSE_id_sim),idx);	
135		
136	fprintf('\nBest simulation model on id:\n');	
137	fprintf('na = nb = %d\n',id_best_na_sim);	
138	fprintf('m = %d\n',id_best_m_sim);	
139	fprintf('MSE = %.6f\n',minAMSE_id_sim);	
140		
141	%% Compare plot on identification with the best fit (SIMULATION CRITERION)	
142	figure(1);	
143	compare(id,id_pred_optimal_s,id_sim_optimal_s);	
144	title("Nonlinear ARX model on identification - simulation criterion");	
145		

146	%% Best model on identification (PREDICTION CRITERION)	
147	% (cele mai bune combinatii de na,nb,m dupa MSE_pred)	
148		
149	[minAMSE_id_pred,idx]=min(AMSE_id_pred(:));	
150	[id_best_na_pred,id_best_m_pred]=ind2sub(size(AMSE_id_pred),idx);	
151		
152	fprintf('\nBest prediction model on id:\n');	
153	fprintf('na = nb = %d\n',id_best_na_pred);	
154	fprintf('m = %d\n',id_best_m_pred);	
155	fprintf('MSE = %.6f\n',minAMSE_id_pred);	
156		
157	%% Compare plot on identification with the best fit (PREDICTION CRITERION)	
158	figure(2);	
159	compare(id,id_pred_optimal_p);	
160	title("Nonlinear ARX model on identification - prediction criterion");	

163	%% VALIDATION	
164		
165		
166	%% Best model on validation (SIMULATION CRITERION)	
167	% (cele mai bune combinatii de na,nb,m dupa MSE_sim)	
168		
169	[minAMSE_val_sim,idx]=min(AMSE_val_sim(:));	
170	[val_best_na_sim,val_best_m_sim]=ind2sub(size(AMSE_val_sim),idx);	
171		
172	fprintf('\nBest simulation model on val:\n');	
173	fprintf('na = nb = %d\n',val_best_na_sim);	
174	fprintf('m = %d\n',val_best_m_sim);	
175	fprintf('MSE = %.6f\n',minAMSE_val_sim);	
176		
177	%% Compare plot on validation with the best fit (SIMULATION CRITERION)	
178	figure(1);	
179	compare(val,val_pred_optimal_s,val_sim_optimal_s);	
180	title("Nonlinear ARX model on validation - simulation criterion");	
181		

182	%% Best model on validation (PREDICTION CRITERION)	
183	% (cele mai bune combinatii de na,nb,m dupa MSE_pred)	
184		
185	[minAMSE_val_pred,idx]=min(AMSE_val_pred(:));	
186	[val_best_na_pred,val_best_m_pred]=ind2sub(size(AMSE_val_pred),idx);	
187		
188	fprintf('\nBest prediction model on val:\n');	
189	fprintf('na = nb = %d\n',val_best_na_pred);	
190	fprintf('m = %d\n',val_best_m_pred);	
191	fprintf('MSE = %.6f\n',minAMSE_val_pred);	
192		
193	%% Compare plot on validation with the best fit (PREDICTION CRITERION)	
194	figure(2);	
195	compare(val,val_pred_optimal_p,val_sim_optimal_p);	
196	title("Nonlinear ARX model on validation - prediction criterion");	
197		

```
202 %% Comparare AMSE on validation
203
204 % on simulation
205 figure(1);
206 hold on;
207 for m=1:max_m
208     plot(1:max_na,AMSE_val_sim(:,m),'-o', 'LineWidth',1.5);
209 end
210 grid on;
211 xlabel('na = nb');
212 ylabel('Simulation AMSE');
213 legend(arrayfun(@(x) sprintf('m = %d', x), 1:max_m, 'UniformOutput', false));
214 title('Simulation AMSE vs model degree');
215
216 % on prediction
217 figure(2);
218 hold on;
219 for m=1:max_m
220     plot(1:max_na,AMSE_val_pred(:,m),'-o','LineWidth',1.5);
```

221	end	
222	grid on;	
223	xlabel('na = nb');	
224	ylabel('Prediction AMSE');	
225	legend(arrayfun(@(x) sprintf('m = %d', x), 1:max_m, 'UniformOutput', false));	
226	title('Prediction AMSE vs model degree');	
227		
228	%% AMSE MAP acelasi lucru da ii imagine colorata(nu arata pre bine)	
229	figure	
230	imagesc(1:max_m,1:max_na,AMSE_val_sim)	
231	xlabel('Polynomial degree m')	
232	ylabel('na = nb')	
233	colorbar	
234	title('Simulation AMSE')	
235		
236	figure	
237	imagesc(1:max_m,1:max_na,AMSE_val_pred)	
238	xlabel('Polynomial degree m')	
239	ylabel('na = nb')	

240	colorbar	
241	title('Prediction AMSE')	
242		
243	%% Acelasi lucru dar este suprafata (nu arata prea bine)	
244		
245	[MM,NA]= meshgrid(1:max_m,1:max_na);	
246		
247	figure(3);	
248	surf(MM,NA,AMSE_val_sim);	
249	xlabel('Polynomial degree m');	
250	ylabel('na = nb');	
251	zlabel('Simulation AMSE');	
252	title('Simulation AMSE surface');	
253	shading interp;	
254		
255	figure(4);	
256	surf(MM,NA,AMSE_val_pred);	
257	xlabel('Polynomial degree m');	
258	ylabel('na = nb');	
259	zlabel('Prediction AMSE');	

```
260 title('Prediction AMSE surface');
261 shading interp;
262
263 %% Comparare MSE identification vs validation (SIMULATION)
264 % pentru cel mai bun caz de pe validare
265
266 na=val_best_na_sim;
267 nb=val_best_na_sim;
268 m=val_best_m_sim;
269
270 Phi_id_best=[];
271 for k=1:N_id
272     phi_k=phi_arx_generator(y_id,u_id,k,na,nb,m);
273     Phi_id_best=[Phi_id_best; phi_k];
274 end
275 theta_best=Phi_id_best\y_id(:);
276
```

```
277 y_id_pred_best=zeros(N_id,1);
278 for k=1:N_id
279     phi_k=phi_arx_generator(y_id,u_id,k,na,nb,m);
280     y_id_pred_best(k)=phi_k*theta_best;
281 end
282
283 y_val_sim_best=zeros(N_val,1);
284 for k=1:N_val
285     phi_k=phi_arx_generator(y_val_sim_best,u_val,k,na,nb,m);
286     y_val_sim_best(k)=phi_k*theta_best;
287 end
288
289 MSE_vector_id=(y_id_pred_best-y_id).^2;
290 MSE_vector_sim=(y_val_sim_best-y_val).^2;
291
292 figure(1);
293 hold on;
294 grid on;
295
```

```
296 plot(1:max_na,AMSE_id_sim(:,val_best_m_sim),'r-o','LineWidth',1.5);
297 plot(1:max_na,AMSE_val_sim(:,val_best_m_sim),'b-o','LineWidth',1.5);
298 legend('MSE on identification','MSE on validation');
299 xlabel('Model order na = nb');
300 ylabel('AMSE');
301 title('MSE identification and validation for simulation');
302
```

```
303 %% Comparare MSE identification vs validation (PREDICTION)
304 % pentru cel mai bun caz de pe validare
305
306 na=val_best_na_pred;
307 nb=val_best_na_pred;
308 m=val_best_m_pred;
309
310 Phi_id_best=[];
311 for k=1:N_id
312     phi_k=phi_arx_generator(y_id,u_id,k,na,nb,m);
313     Phi_id_best=[Phi_id_best; phi_k];
314 end
```

```
315 theta_best=Phi_id_best\y_id(:);
316
317 y_id_pred_best=zeros(N_id,1);
318 for k = 1:N_id
319     phi_k=phi_arx_generator(y_id,u_id,k,na,nb,m);
320     y_id_pred_best(k)=phi_k*theta_best;
321 end
322
323 y_val_pred_best=zeros(N_val,1);
324 for k=1:N_val
325     phi_k=phi_arx_generator(y_val,u_val,k,na,nb,m);
326     y_val_pred_best(k)=phi_k*theta_best;
327 end
328
329 MSE_vector_id=(y_id_pred_best-y_id).^2;
330 MSE_vector_val=(y_val_pred_best-y_val).^2;
331
332 figure(2);
333 hold on;
334 grid on;
```

```
335 plot(1:max_na,AMSE_id_pred(:,val_best_m_pred),'r-o','LineWidth',1.5);
336 plot(1:max_na,AMSE_val_pred(:,val_best_m_pred),'b-o','LineWidth',1.5);
337 legend('MSE on identification','MSE on validation');
338 xlabel('Model order na = nb');
339 ylabel('AMSE');
340 title('MSE identification and validation for prediction');
341
```

```
342 %% Functii generare phi
```

```
343
344 function phi_k=phi_arx_generator(y,u,k,na,nb,m)
345
346     d=zeros(1,na+nb);
347     for i=1:na
348         if k-i>0
349             d(i)=y(k-i);
350         end
351     end
```

```
353     for j=1:nb
354         if k-j>0
355             d(na+j)=u(k-j);
356         end
357     end
358     phi_k=[1 d];
359
360     if m>=2
361         for i=1:length(d)
362             for j=i:length(d)
363                 phi_k=[phi_k d(i)*d(j)];
364             end
365         end
366     end
```

```

368     if m>=3
369         for i=1:length(d)
370             for j=i:length(d)
371                 for l=j:length(d)
372                     phi_k=[phi_k d(i)*d(j)*d(l)];
373                 end
374             end
375         end
376     end
377
378 end
379
380 % generarea unui tabel de exponenți
381 function pow=powers_generator(p,m)
382     pow=zeros(1,p);
383     for i=1:p
384         e=zeros(1,p);
385         e(i)=1;
386         pow=[pow;e];
387     end

```

```
388     if m>=2
389         for i=1:p
390             for j=i:p
391                 e=zeros(1,p); e(i)=e(i)+1; e(j)=e(j)+1;
392                 pow=[pow;e];
393             end
394         end
395     end
396     if m>=3
397         for i=1:p
398             for j=i:p
399                 for k=j:p
400                     e=zeros(1,p); e(i)=e(i)+1; e(j)=e(j)+1; e(k)=e(k)+1;
401                     pow=[pow;e];
402                 end
403             end
404         end
405     end
406 end
```

```
408 % generarea phi cu ajutorul tabelului de exponenți
409 % mai bun pentru alte valori ale lui m
410 function phi_k=powers_phi_arx_generator(y,u,k,na,nb,pow)
411
412     d=zeros(1,na+nb);
413     for i=1:na
414         if (k-i)>0
415             d(i)=y(k-i);
416         end
417     end
418
419     for j=1:nb
420         if (k-j)>0
421             d(na+j)=u(k-j);
422         end
423     end
424
425     phi_k=zeros(1,size(pow,1));
426     for r=1:size(pow,1)
427         phi_k(r)=prod(d.^pow(r,:));
428     end
429
430 end
```